

Appendix C: IEEE 830 Template

Software Requirements Specification

for

Gym Management System

Version 0.2

Prepared by Jaime Esteban Reaño

IFE – Software Engineering

11/03/2026

«Page Break»

Table of Contents

«Table of Contents»

Revision History

Jaime Esteban Reaño	04/03/2026	Initial version of the SRS document created. Project topic selected: Gym Management System.	0.1
Jaime Esteban Reaño	11/03/2026	Added Introduction and Overall Description sections up to 2.1 based on requirements analysis.	0.2

«Page break»

1. Introduction

1.1. Purpose

This document specifies the software requirements for the Gym Management System, a software solution intended to support the daily management of a gym. The system will provide functionalities for member registration, class booking, trainer management, membership tracking, and payment control. This specification defines the current scope of the project and will be progressively updated as new analysis and design artifacts are developed during the course.

1.2. Document Conventions

Functional requirements are identified using the prefix FR, while non-functional requirements are identified using the prefix NFR. All mandatory requirements are written using the word “shall” to ensure precision and verifiability. UML diagrams included in the document are used as analysis and design support artifacts.

1.3. Intended Audience and Reading Suggestions

This document is intended for course instructors, project team members, testers, and future developers of the Gym Management System. Readers should begin with the Introduction and overall description sections to understand the purpose and scope of the project, and then continue with the system features and interface requirements

1.4. Product Scope

This Gym Management System aims to digitize and simplify the management of gym operations. Its main objective is to centralize information related to members, trainers, fitness classes, memberships, and payments in a single system. The expected benefits include improved administrative efficiency, reduced manual errors, easier scheduling, and better service for gym members

1.5. References

- Lab 1: Requirements Analysis – Software Engineering course materials
- Lab 2: UML Use Case Diagrams – Software Engineering course materials
- Lab 3: UML CLASS Diagrams – Software Engineering course materials

2. Overall Description

2.1. Product Perspective

This Gym Management System is conceived as a new standalone software product. It is intended to replace informal or manual management methods based on spreadsheets, paper records, or disconnected applications. The system will centralize core gym operations in a single platform.

2.2. Product Features

- Member registration and profile management
- Membership plan management
- Trainer management
- Class scheduling and booking
- Payment tracking and membership validation
- Attendance monitoring

2.3. User Classes and Characteristics

The main user classes of the Gym Management System are Administrators, Trainers, and Members. Administrators are the primary management users and have the highest privilege level, being responsible for registering members, managing classes, assigning trainers, and controlling payments. Trainers use the system to consult assigned sessions, review schedules, and possibly track attendance. Members are end users who interact with the system to manage their personal information, check available classes, make reservations, and review membership status.

2.4. Operating Environment

This system is expected to operate as a web – based application. You could access it through modern browsers on desktop and also mobile devices. It should be compatible with common operating systems, like Windows, macOS, Android and even iOS , through browser access. However, an internet connection is required for its normal operation.

2.5. Design and Implementation Constraints

The project is developed in an academic environment and therefore its scope is limited to the core functionalities of a gym management platform. The system should prioritize clarity, usability, and modularity over advanced

commercial features. Personal data related to members and trainers must be handled securely. The specification and design of the project will evolve incrementally according to the software engineering laboratory activities carried out during the course.

2.6. User Documentation

The system is expected to include basic user documentation adapted to an academic project context. This documentation may include a short administrator guide explaining how to register members, manage classes, assign trainers, and record payments, as well as a brief user guide for members describing how to view classes, make reservations, and review membership information. If implemented, online help or embedded interface hints may also be provided. The documentation will be delivered in digital format, most likely as PDF files and brief in-system textual guidance.

2.7. Assumptions and Dependencies

The specification assumes that the gym operates with clearly defined user roles, including administrators, trainers, and members. It is also assumed that the system will be accessed through devices with a stable internet connection and a compatible web browser. The project depends on decisions that may be taken in later stages of development, such as the database solution, the authentication mechanism, and the deployment environment. Since the project is developed incrementally in parallel with the laboratory activities of the course, some requirements and design decisions may evolve as new software engineering concepts are introduced.

3. System Features

«The purpose here is to give a brief introduction to the system features, so that §4 has something to reference. Then, the full explanation of the system features is given in §5, likely referencing §4. The numbering in §3 and §5 should match.»

3.1. «Feature Name»

«Brief description of the feature, generally 1-3 sentences»

3.«N». «Additional feature blocks as needed»

4. External Interface Requirements

4.1. User Interfaces Overview

«Describe the high level functionality of the system from the user's perspective. Explain how the user should be able to use your system to complete all the expected features and the feedback information that will be displayed for the user (for each screen the user will see). For example, specify whether it is GUI or text based interface and how at high level it would work. Discuss at high level how user will interact with the game such as clicking buttons, typing in some selection character, etc.»

«Describe the minimum requirements for the interface. This may include some sample screen images whether for text or GUI approach, any GUI standards or product family style guides that are to be followed, screen layout constraints, for GUI standard buttons and functions (e.g., help) that must appear on every screen, keyboard shortcuts, error message display standards, and so on»

4.2. Hardware Interfaces

«Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.»

4.3. Software Interfaces

«Describe the connections between this product and other specific software components (name and version), including databases, interpreters, operating systems, tools, libraries, and integrated commercial components.

Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.»

5. System Features/Modules

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

5.1. «System Feature Name»

5.1.1 Description and Priority

«Provide a short description of the feature. Give its priority, too.»

5.1.2 Stimulus/Response Sequences

«"Stimulus" or "Condition": «Describe the stimulus or condition»

Response: «Describe the response»

«Additional Stimulus/Response pairs as needed»

5.1.3 Functional Requirements

«Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.»

REQ-1.1: «Some sentence goes here, in "shall" form, usually something like this: "Upon <some event or condition>, the <system, or the name of a module> shall <perform some action>." Note that the entire requirement must be stated in the requirements area. Everything else is just a lead-up to this. There should be one such entry per S/R pair.»

REQ-1.«N»: «Additional S/R-related requirements as needed.»

REQ-1.«N»: «List any additional requirements needed in "Shall" form, but not necessarily in "Upon/Shall" form.»

REQ-1.«N»: «Additional requirements as needed.»

5.«N». «Additional feature blocks»

6. Nonfunctional Requirements

6.1. Performance

NF-1.1: «Any performance-related issue»

NF-1.«N»: «Any performance-related issue»

6.2. Security

NF-2.1: «Any security-related issue»

NF-2.«N»: «Any security-related issue»

6.«N». «Any additional “nonfunctional” blocks»